

MATH3881/5881: Statistical Machine Learning Theory

Week 2: Theory of Feedforward Neural Networks: Exercises and Solutions

Sarat Moka

UNSW, Sydney

Exercise 2.1.1 (Forward Pass in a ReLU Network)

Consider a network with $p = 2$ inputs, one hidden layer of width $N_1 = 3$, and $q = 1$ output, with ReLU hidden activations and identity output activation.

(a) Write down the dimensions of all weight matrices and bias vectors, and verify the total parameter count using the formula $|\theta| = \sum_{\ell=1}^L N_{\ell}(N_{\ell-1} + 1)$.

(b) Given

$$W^{[1]} = \begin{pmatrix} 1 & -1 \\ 0 & 2 \\ 1 & 0 \end{pmatrix}, \quad b^{[1]} = \mathbf{0}, \quad W^{[2]} = \begin{pmatrix} 1 & 1 & -1 \end{pmatrix}, \quad b^{[2]} = 0,$$

compute the output $f_{\theta}(x)$ for $x = (1, 1)^{\top}$.

(c) What is $f_{\theta}(x)$ for $x = (-2, 1)^{\top}$? Explain the role of the ReLU activations in this case.

Solution. (a) The network has $L = 2$ layers ($N_0 = 2$, $N_1 = 3$, $N_2 = 1$), so:

$$W^{[1]} \in \mathbb{R}^{3 \times 2}, \quad b^{[1]} \in \mathbb{R}^3, \quad W^{[2]} \in \mathbb{R}^{1 \times 3}, \quad b^{[2]} \in \mathbb{R}^1.$$

Parameter count: $N_1(N_0 + 1) + N_2(N_1 + 1) = 3(2 + 1) + 1(3 + 1) = 9 + 4 = \mathbf{13}$.

(b) For $x = (1, 1)^{\top}$:

$$z^{[1]} = W^{[1]}x = \begin{pmatrix} 1 - 1 \\ 0 + 2 \\ 1 + 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 2 \\ 1 \end{pmatrix}, \quad a^{[1]} = \sigma_{\text{ReLU}}(z^{[1]}) = \begin{pmatrix} 0 \\ 2 \\ 1 \end{pmatrix}.$$

$$z^{[2]} = W^{[2]}a^{[1]} = 1 \cdot 0 + 1 \cdot 2 + (-1) \cdot 1 = 1, \quad f_{\theta}(x) = z^{[2]} = \boxed{1}.$$

(c) For $x = (-2, 1)^{\top}$:

$$z^{[1]} = W^{[1]}x = \begin{pmatrix} -2 - 1 \\ 0 + 2 \\ -2 + 0 \end{pmatrix} = \begin{pmatrix} -3 \\ 2 \\ -2 \end{pmatrix}, \quad a^{[1]} = \sigma_{\text{ReLU}}(z^{[1]}) = \begin{pmatrix} 0 \\ 2 \\ 0 \end{pmatrix}.$$

$$z^{[2]} = W^{[2]}a^{[1]} = 1 \cdot 0 + 1 \cdot 2 + (-1) \cdot 0 = 2, \quad f_{\theta}(x) = \boxed{2}.$$

ReLU zeroes out neurons 1 and 3, whose pre-activations (-3 and -2) are negative. Only neuron 2 contributes to the output. This illustrates how ReLU creates a sparse, input-dependent activation

pattern: different inputs activate different subsets of neurons, giving the network its piecewise-linear character.

Exercise 2.2.1 (Properties of Activation Functions)

- (a) Verify the identity $\sigma_{\text{Tanh}}(u) = 2\sigma_{\text{Sig}}(2u) - 1$. What does this imply about the relationship between networks using tanh and sigmoid activations?
- (b) Show that σ_{ReLU} is 1-Lipschitz, i.e. $|\sigma_{\text{ReLU}}(u) - \sigma_{\text{ReLU}}(v)| \leq |u - v|$ for all $u, v \in \mathbb{R}$.
- (c) Verify the derivative formula $\dot{\sigma}_{\text{Sig}}(u) = \sigma_{\text{Sig}}(u)(1 - \sigma_{\text{Sig}}(u))$. Find the maximum value of $\dot{\sigma}_{\text{Sig}}$ and explain its relevance to the vanishing gradient problem.

Solution. (a) Starting from the right-hand side:

$$2\sigma_{\text{Sig}}(2u) - 1 = \frac{2}{1 + e^{-2u}} - 1 = \frac{2 - (1 + e^{-2u})}{1 + e^{-2u}} = \frac{1 - e^{-2u}}{1 + e^{-2u}}.$$

Multiplying numerator and denominator by e^u :

$$= \frac{e^u - e^{-u}}{e^u + e^{-u}} = \sigma_{\text{Tanh}}(u).$$

Implication: Any network with tanh hidden activations can be simulated by a network with sigmoid activations (with weights and biases rescaled by 2), and vice versa. The two activation functions give equivalent expressive power; the choice is mainly one of numerical convenience.

(b) Let $u, v \in \mathbb{R}$. Denote $f = \sigma_{\text{ReLU}}$. We consider four cases:

- Both $u, v \geq 0$: $|f(u) - f(v)| = |u - v|$.
- Both $u, v < 0$: $|f(u) - f(v)| = 0 \leq |u - v|$.
- $u \geq 0, v < 0$: $|f(u) - f(v)| = u \leq u - v = |u - v|$ (since $-v > 0$).
- $u < 0, v \geq 0$: $|f(u) - f(v)| = v \leq v - u = |u - v|$ (since $-u > 0$).

In every case $|\sigma_{\text{ReLU}}(u) - \sigma_{\text{ReLU}}(v)| \leq |u - v|$, so σ_{ReLU} is 1-Lipschitz.

(c) By the quotient rule:

$$\frac{d}{du}\sigma_{\text{Sig}}(u) = \frac{e^{-u}}{(1 + e^{-u})^2} = \frac{1}{1 + e^{-u}} \cdot \frac{e^{-u}}{1 + e^{-u}} = \sigma_{\text{Sig}}(u) \cdot \left(1 - \frac{1}{1 + e^{-u}}\right) = \sigma_{\text{Sig}}(u)(1 - \sigma_{\text{Sig}}(u)).$$

Setting $g(u) = \sigma_{\text{Sig}}(u)(1 - \sigma_{\text{Sig}}(u))$ and differentiating: $g'(u) = \dot{\sigma}_{\text{Sig}}(u)(1 - 2\sigma_{\text{Sig}}(u))$. Since $\dot{\sigma}_{\text{Sig}}(u) > 0$ always, $g'(u) = 0 \Leftrightarrow \sigma_{\text{Sig}}(u) = \frac{1}{2} \Leftrightarrow u = 0$. Hence the maximum is

$$\dot{\sigma}_{\text{Sig}}(0) = \frac{1}{2}\left(1 - \frac{1}{2}\right) = \frac{1}{4}.$$

Relevance to vanishing gradients: Each sigmoid layer contributes a factor of at most $\frac{1}{4}$ to the gradient. In an L -layer network the gradient reaching the first layer involves a product of L such factors, giving magnitude at most $(1/4)^L$, which decays exponentially with depth. This makes it very hard to train early layers of deep sigmoid networks.

Exercise 2.4.1 (Identity Collapse)

Verify the identity-collapse formula for $L = 3$ by expanding the composition $W^{[3]}(W^{[2]}(W^{[1]}x + b^{[1]}) + b^{[2]}) + b^{[3]}$ directly, and confirm that your $\tilde{b}$ matches the closed-form expression $\tilde{b} = \sum_{\ell=1}^L (\prod_{\tilde{\ell}=\ell+1}^L W^{[\tilde{\ell}]})b^{[\ell]}$.

Solution. With identity activations at all three layers (so $a^{[\ell]} = z^{[\ell]}$ for $\ell = 1, 2, 3$, and the output $f_{\theta}(x) = z^{[3]}$), expand layer by layer:

$$\begin{aligned} a^{[1]} &= W^{[1]}x + b^{[1]}, \\ a^{[2]} &= W^{[2]}a^{[1]} + b^{[2]} = W^{[2]}W^{[1]}x + W^{[2]}b^{[1]} + b^{[2]}, \\ f_{\theta}(x) = z^{[3]} &= W^{[3]}a^{[2]} + b^{[3]} = W^{[3]}W^{[2]}W^{[1]}x + W^{[3]}W^{[2]}b^{[1]} + W^{[3]}b^{[2]} + b^{[3]}. \end{aligned}$$

Setting $\tilde{W} = W^{[3]}W^{[2]}W^{[1]}$, the network reduces to a single affine map $f_{\theta}(x) = \tilde{W}x + \tilde{b}$ with

$$\tilde{b} = W^{[3]}W^{[2]}b^{[1]} + W^{[3]}b^{[2]} + b^{[3]}.$$

This matches the closed-form expression at $L = 3$: writing out $\tilde{b} = \sum_{\ell=1}^3 (\prod_{\tilde{\ell}=\ell+1}^3 W^{[\tilde{\ell}]})b^{[\ell]}$ gives

$$\underbrace{W^{[3]}W^{[2]}b^{[1]}}_{\ell=1} + \underbrace{W^{[3]}b^{[2]}}_{\ell=2} + \underbrace{b^{[3]}}_{\ell=3},$$

where the empty product at $\ell = L = 3$ is the identity. Regardless of the individual matrices, three identity-activation layers behave as a single linear layer.

Exercise 2.5.1 (Variance Propagation and Xavier Initialisation)

- Derive the variance propagation formula. Starting from $z_i^{[\ell]} = \sum_{j=1}^{N_{\ell-1}} w_{i,j}^{[\ell]} a_j^{[\ell-1]}$ (ignoring bias), and assuming $w_{i,j}^{[\ell]}$ and $a_j^{[\ell-1]}$ are independent with zero mean and $\text{Var}(w_{i,j}^{[\ell]}) = \check{w}_{\ell}$, show that $\text{Var}(z_i^{[\ell]}) = N_{\ell-1} \check{w}_{\ell} \text{Var}(a_j^{[\ell-1]})$.
- For a network with $L = 4$ layers and widths $(N_0, N_1, N_2, N_3, N_4) = (10, 8, 6, 4, 1)$ using Xavier initialisation $\text{Var}(w_{i,j}^{[\ell]}) = 1/N_{\ell-1}$, compute $\text{Var}(w_{i,j}^{[\ell]})$ for each layer $\ell = 1, \dots, 4$.

Solution. (a) Since the weights $w_{i,j}^{[\ell]}$ and activations $a_j^{[\ell-1]}$ are independent and zero-mean, products $w_{i,j}^{[\ell]} a_j^{[\ell-1]}$ across different j are uncorrelated. Therefore:

$$\text{Var}(z_i^{[\ell]}) = \sum_{j=1}^{N_{\ell-1}} \text{Var}(w_{i,j}^{[\ell]} a_j^{[\ell-1]}).$$

For independent zero-mean random variables X, Y : $\text{Var}(XY) = \mathbb{E}[X^2Y^2] - (\mathbb{E}[XY])^2 = \mathbb{E}[X^2]\mathbb{E}[Y^2] - 0 = \text{Var}(X) \text{Var}(Y)$. Hence:

$$\text{Var}(w_{i,j}^{[\ell]} a_j^{[\ell-1]}) = \text{Var}(w_{i,j}^{[\ell]}) \text{Var}(a_j^{[\ell-1]}) = \check{w}_\ell \text{Var}(a_j^{[\ell-1]}).$$

Summing over $j = 1, \dots, N_{\ell-1}$ (all terms identical by the assumed common variance):

$$\text{Var}(z_i^{[\ell]}) = N_{\ell-1} \check{w}_\ell \text{Var}(a_j^{[\ell-1]}).$$

Setting $N_{\ell-1} \check{w}_\ell = 1$ ensures variance is preserved across layers, giving Xavier initialisation $\check{w}_\ell = 1/N_{\ell-1}$.

(b) Applying $\text{Var}(w_{i,j}^{[\ell]}) = 1/N_{\ell-1}$ to the given architecture:

Layer ℓ	Fan-in $N_{\ell-1}$	$\text{Var}(w_{i,j}^{[\ell]})$
1	$N_0 = 10$	$1/10 = 0.1$
2	$N_1 = 8$	$1/8 = 0.125$
3	$N_2 = 6$	$1/6 \approx 0.167$
4	$N_3 = 4$	$1/4 = 0.25$

Layers closer to the output (smaller fan-in) use larger weight variances to maintain the signal magnitude as the layer width narrows.

Exercise 2.6.1 (Weight Decay and Dropout)

- (a) Show directly that the weight decay update $\theta^{(t+1)} = (1 - \alpha\lambda) \theta^{(t)} - \alpha \nabla C(\theta^{(t)})$ is exactly one step of gradient descent on the regularised objective $\tilde{C}(\theta) = C(\theta) + \frac{\lambda}{2} \|\theta\|^2$ with learning rate α .
- (b) For dropout with keep probability $p_{\text{keep}}^{[\ell]}$, consider the activation-weighted sum entering neuron j of layer $\ell + 1$ during training,

$$s_j := \sum_{i \text{ kept}} w_{i,j}^{[\ell+1]} a_i^{[\ell]},$$

so that the pre-activation is $z_j^{[\ell+1]} = b_j^{[\ell+1]} + s_j$ (biases are not subject to dropout). Show that $\mathbb{E}[s_j]$ equals $p_{\text{keep}}^{[\ell]}$ times the corresponding sum without dropout. Then verify that inverted dropout corrects for this so that the expected training pre-activation matches the no-dropout pre-activation.

Solution. (a) Gradient descent on $\tilde{C}(\theta) = C(\theta) + \frac{\lambda}{2} \|\theta\|^2$ with step α :

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla \tilde{C}(\theta^{(t)}) = \theta^{(t)} - \alpha (\nabla C(\theta^{(t)}) + \lambda \theta^{(t)}) = (1 - \alpha\lambda) \theta^{(t)} - \alpha \nabla C(\theta^{(t)}).$$

The L2 penalty contributes the multiplicative factor $(1 - \alpha\lambda)$, which shrinks (*decays*) the weights at every step independently of the gradient.

(b) Let $D_i \in \{0, 1\}$ be a Bernoulli random variable with $\mathbb{P}(D_i = 1) = p_{\text{keep}}^{[\ell]}$, indicating whether neuron i is kept. The activation-weighted sum during training is

$$s_j = \sum_{i=1}^{N_\ell} D_i w_{i,j}^{[\ell+1]} a_i^{[\ell]}.$$

Taking expectation (conditional on $a^{[\ell]}$, using $\mathbb{E}[D_i] = p_{\text{keep}}^{[\ell]}$ and the linearity of expectation):

$$\mathbb{E}[s_j] = \sum_{i=1}^{N_\ell} \mathbb{E}[D_i] w_{i,j}^{[\ell+1]} a_i^{[\ell]} = p_{\text{keep}}^{[\ell]} \sum_{i=1}^{N_\ell} w_{i,j}^{[\ell+1]} a_i^{[\ell]} = p_{\text{keep}}^{[\ell]} \cdot s_j(\text{no dropout}).$$

So the expected activation-weighted sum is scaled down by $p_{\text{keep}}^{[\ell]}$ relative to the no-dropout value. Adding the bias (which is unaffected by dropout), the expected pre-activation in training is $\mathbb{E}[z_j^{[\ell+1]}] = b_j^{[\ell+1]} + p_{\text{keep}}^{[\ell]} \cdot s_j(\text{no dropout})$, which differs from the no-dropout pre-activation $b_j^{[\ell+1]} + s_j(\text{no dropout})$ unless $p_{\text{keep}}^{[\ell]} = 1$.

Inverted dropout correction: Scale kept activations by $1/p_{\text{keep}}^{[\ell]}$, giving the corrected sum

$$\tilde{s}_j = \sum_{i=1}^{N_\ell} \frac{D_i}{p_{\text{keep}}^{[\ell]}} w_{i,j}^{[\ell+1]} a_i^{[\ell]}.$$

Then $\mathbb{E}[\tilde{s}_j] = (1/p_{\text{keep}}^{[\ell]}) \cdot p_{\text{keep}}^{[\ell]} \cdot s_j(\text{no dropout}) = s_j(\text{no dropout})$, and so the expected pre-activation $\mathbb{E}[b_j^{[\ell+1]} + \tilde{s}_j] = b_j^{[\ell+1]} + s_j(\text{no dropout})$ matches the no-dropout pre-activation exactly. This is why the trained weight matrices can be used *unchanged* at test time.